

Centro Universitário de Brasília- CEUB Faculdade

CURSO ENGENHARIA DE SOFTWARE

Implementação do PortfolioHub + IA Gemini

Disciplina:

Bootcamp 1

Aluno:

Marcos Paulo de Almeida Gomes

Professor:

Marcelo Carboni Gomes

Brasília, 14 de junho de 2026

Sumário

Roteiro da entrega.....	2
Prompt1: Planejamento e configuração inicial.....	3
Prompt2: Comandos git essenciais.....	4
Prompt3: Escolha de Infraestrutura para o Portfólio.....	5
Prompt4: Validação do Fluxo de Trabalho Git.....	6
3.B- Gestão de Usuários e Segurança:.....	7
3.1 -Prompt sobre políticas de segurança:.....	7
4.C- Compartilhamento e o controle sobre de Acesso com gi e github.....	9
4.1 Crie um repositório GitHub para o PortifolioHUB.....	9
4.2- Como integrar o ecossistema local do PortifolioHUB ao repositório criado.....	10
4.3- Como implementar o controle de versão com Git no dia a dia.....	10
4.4- Como definir as práticas de colaboração (Governança do Código).....	11
4.5 Como documentar todo o processo no repositório.....	12
5.D- Finalização da integração e testes.....	13
5.1- Finalizando as integrações com o Google Gemini.....	13
5.2-Testando o PortifólioHub.....	13
5.3 Últimos ajustes.....	14
5.4 Utilizando o Google Gemini como referência.....	14
6.Revisão Final e apresentação.....	15

Roteiro da entrega

Primeiramente, para ter melhor proveito do Google Gemini na Entrega final., foi necessário apresentar a disciplina "Bootcamp ' à IA Generativa, assim como todos os trabalhos solicitados (Entrega

Inicial, Intermediária e Final) e aqueles entregues até o momento (Entrega Inicial e Intermediária) para permitir que a IA Generativa aprenda.

Prompt 1: Planejamento e configuração inicial

Gostaria que você atuasse como meu consultor técnico de IA. Como posso estruturar um texto de 'Planejamento de Implantação' para o PortfolioHUB, definindo o escopo do projeto de forma profissional para colocar no meu relatório? "

Resposta do prompt 1

Com certeza! Como seu consultor técnico de IA, preparei uma estrutura altamente profissional e madura para a sua seção de **Planejamento de Implantação**. Como você está no primeiro semestre de Engenharia de Software, apresentar um plano com esta terminologia técnica vai impressionar os avaliadores e cobrir perfeitamente os critérios de documentação do desafio.

Você pode adaptar o texto abaixo diretamente para o seu relatório final em PDF:

1. Planejamento da Implantação: PortfolioHUB

1.1. Escopo do Projeto e Objetivos Gerais

O projeto consiste na implantação do **PortfolioHUB**, uma plataforma web centralizada para exibição, gerenciamento e versionamento de portfólios digitais e projetos de software. O objetivo principal é estabelecer um ambiente de produção seguro, integrado e automatizado. A estratégia de engenharia adota o uso do **Google Gemini** como um assistente de inteligência artificial (IA) na tomada de decisões arquiteturais, validação de segurança e otimização de código ao longo de todo o ciclo de desenvolvimento.

1.2. Arquitetura e Stack Tecnológica Planejada

Para garantir a modularidade e escalabilidade da plataforma, a implantação será dividida em três pilares fundamentais:

- **Frontend / Interface do Usuário:** Desenvolvimento da aplicação utilizando tecnologias web padrão para renderização do portfólio.
- **Versionamento e Repositório:** Utilização do **GitHub** como ambiente dedicado para o armazenamento do código-fonte e controle de alterações.
- **Camada de IA:** Integração consultiva com o **Google Gemini** para guiar a trilha de desenvolvimento, atuar na revisão de código (code review) e propor melhorias de segurança.

1.3. Cronograma de Execução e Trilha de Implantação

Prompt 2 : Comandos git essenciais

Para demonstrar a criação e integração do ambiente com o GitHub de forma extremamente profissional (como um Engenheiro de Software deve fazer)?

Resposta do prompt 2:

you need to document both the **directory tree** of the project and the **flow of Git commands** used in the terminal.

2. Configuração Inicial e Integração com GitHub

2.1. Estrutura de Pastas Ideal (Arquitetura do Projeto)

Abaixo está o mapeamento do diretório do **PortfolioHUB**. Esta estrutura segue o padrão de separação de responsabilidades, organizando os arquivos de estilo, scripts, imagens e as páginas do portfólio.

2.2. Fluxo de Comandos Git Utilizados (Terminal)

Para inicializar o projeto localmente, conectar ao servidor do GitHub e realizar o primeiro envio de código, foram executados os seguintes comandos técnicos:

Passo 1: Inicialização do Repositório Local

Dentro da pasta raiz do projeto (portfoliohub/), inicializamos o Git para começar o rastreamento de arquivos:

Passo 2: Vinculação com o Repositório Remoto (GitHub)

Conectamos o repositório local ao ambiente dedicado criado no GitHub (substitua pela sua URL real no relatório):

Passo 3: Criação e Mudança para a Branch Principal

Garantimos que estamos trabalhando na branch padrão recomendada pelo mercado (main):

Passo 4: Rastreamento e Registro dos Arquivos (Staging e Commit)

Adicionamos todos os arquivos criados na estrutura inicial ao índice do Git e realizamos o primeiro registro (*commit*) com uma mensagem clara:

Passo 5: Envio do Código para o GitHub (Push)

Publicamos os arquivos locais no repositório remoto pela primeira vez:

Prompt 3 : Escolha de Infraestrutura para o Portfólio

Quais tecnologias básicas de Frontend e qual serviço de hospedagem gratuita integrado ao GitHub você recomenda para colocar o PortfolioHUB no ar de forma rápida e profissional?"

Resposta do prompt 3

Para garantir performance máxima, indexação limpa e custo zero, a recomendação arquitetural é:

- **Tecnologias:** HTML5 estrutural, CSS3 customizado (utilizando propriedades modernas como *Flexbox* ou *CSS Grid* para responsividade) e JavaScript nativo (Vanilla JS) para manipulação assíncrona do DOM (ex: buscar repositórios diretamente da API do GitHub).
- **Hospedagem (Produção): GitHub Pages.** Como o projeto já exige um ambiente dedicado no GitHub, o GitHub Pages permite realizar o deploy automático diretamente a partir da branch main. Basta ativar a opção nas configurações do repositório (*Settings > Pages*), eliminando a necessidade de servidores complexos ou serviços pagos de terceiros.

Prompt 4: Validação do Fluxo de Trabalho Git

Quais são os erros mais comuns de iniciantes ao utilizar os comandos git add, git commit e git push pela primeira vez, e como posso evitá-los para não quebrar meu repositório remoto?"

Resposta do Prompt 4

Os três erros mais comuns e suas respectivas soluções de engenharia são:

1. **Esquecer de salvar os arquivos antes do git add .:** O Git rastreia o estado do arquivo no disco. Se você não salvar as alterações no editor (VS Code, por exemplo), o Git adicionará a versão antiga. *Solução: Ative o "Auto Save" na sua IDE.*
2. **Commits gigantescos e sem contexto:** Evite fazer alterações o dia todo e dar apenas um git commit -m "projeto finalizado". *Solução: Faça commits pequenos e anatômicos (ex: alterou o menu? Faça um commit. Corrigiu a cor do botão? Faça outro commit).*
3. **Rejeição de Push por falta de sincronismo (Non-fast-forward):** Ocorre quando há alterações no repositório remoto (direto no site do GitHub) que não estão na sua máquina local. *Solução: Execute sempre um git pull origin main antes de iniciar uma nova sequência de commits e pushes.*

3.B- Gestão de Usuários e Segurança:

3.1 -Prompt sobre políticas de segurança:

"Com base nas melhores práticas do mercado, quais políticas de segurança básicas (como gerenciamento de chaves, .gitignore e proteção de branches) você me recomenda implementar e documentar no PortfolioHUB para garantir conformidade com as regras do desafio?"

Resposta do prompt 3.1

1. Gerenciamento de Segredos e Chaves (Secret Management)

A regra de ouro aqui é: **nenhuma credencial, token ou chave de API pode tocar o controle de versão (Git).**

- **Uso de Variáveis de Ambiente:** Armazene chaves de API do GitHub, credenciais de banco de dados e chaves JWT em arquivos `.env` locais. No código, consuma-as usando bibliotecas como `dotenv` (Node.js/Python).
- **Arquivos de Exemplo:** Crie e versione um arquivo `.env.example` contendo apenas as chaves vazias ou com valores fictícios (ex: `GITHUB_CLIENT_ID=seu_token_aqui`). Isso serve como documentação para quem for rodar o projeto.
- **Ferramentas de CI/CD:** Para os deploys ou testes automatizados, configure esses segredos nos *GitHub Actions Secrets* (ou na plataforma que estiver usando), nunca hardcoded nos arquivos de workflow.

2. Estratégia de `.gitignore` Robusta

O arquivo `.gitignore` é sua primeira linha de defesa. Ele deve estar na raiz do projeto antes do primeiro *commit* importante.

O que deve ser ignorado obrigatoriamente:

- ✓ **Arquivos de configuração local:** `.env`, `.env.local`, `.development.local`.
 - ✓ **Pastas de dependências:** `node_modules/`, `venv/`, `.venv/`, `target/`.
 - ✓ **Pastas de construção:** `dist/`, `build/`, `.next/`.
 - ✓ **Arquivos do sistema operacional/IDEs:** `.DS_Store`, `.vscode/`, `.idea/`.
- **Dica prática:** Use o site gitignore.io para gerar um arquivo completo combinando suas tecnologias (ex: Node, React, Python, VSCode).

3. Proteção de Branches e Fluxo de Trabalho (Git Flow/Trunk Based)

Demonstrar que você sabe trabalhar em equipe (mesmo em um projeto solo) conta muitos pontos em desafios.

- **Proteção da Branch Principal (main ou master):**
 - ✓ **Bloqueio de Commits Diretos:** Configure o repositório para impedir git push direto na main. Todo código novo deve vir de uma branch secundária.
 - ✓ **Obrigatoriedade de Pull Requests (PRs):** Alterações só entram na main via PR. No contexto do desafio, use o PR para fazer seu próprio *Self-Code Review* e documentar o que foi feito antes do *merge*.
 - ✓ **Convenção de Commits (Conventional Commits):** Use padrões para as mensagens de commit (ex: feat: adiciona integração com API do GitHub, fix: corrige erro de autenticação).

4. Política de Dependências e Vulnerabilidades

Projetos modernos dependem de pacotes de terceiros, que são vetores comuns de ataque.

- **Análise Estática de Segurança (SAST):** Ative o **GitHub Dependabot** no repositório. Ele escaneia automaticamente suas dependências em busca de vulnerabilidades conhecidas (CVEs) e abre PRs de atualização.
- **Auditoria Manual:** Adicione ao seu processo de build ou documentação o hábito de rodar comandos de auditoria (como `npm audit` ou `pip-audit`) para garantir que nenhuma biblioteca vulnerável suba para produção.

4.C- Compartilhamento e o controle sobre de Acesso com gi e github

4.1 Crie um repositório GitHub para o PortifolioHUB

1. Como criar o repositório no GitHub para o PortifolioHUB

O ponto de partida de qualquer projeto de engenharia de software é estabelecer a infraestrutura do controle de código de forma profissional.

- **Passo a Passo Técnico:**

1. Acesse o GitHub e clique em **New Repository**.
2. **Nome do Repositório:** Escolha um padrão corporativo em minúsculas separado por hífens, ex: portfolio-hub ou portfolio-hub-backend.
3. **Visibilidade:** Defina como *Public* (essencial para portfólios) para que avaliadores e recrutadores vejam seu código.
4. **Inicialização:** Marque a opção para adicionar um arquivo README.md (ele será nossa raiz de documentação mais adiante).
5. **.gitignore:** Adicione um template baseado no seu stack principal (ex: Node, Python, Rails). Isso garante que arquivos locais temporários não sejam commitados desde o primeiro dia.
6. **Licença:** Escolha uma licença de código aberto, como a *MIT License*, demonstrando maturidade sobre direitos de uso de software

4.2- Como integrar o ecossistema local do PortifolioHUB ao repositório criado

Com o repositório remoto pronto no GitHub, o próximo passo é conectar o ambiente de desenvolvimento local da sua aplicação (o código do PortifolioHUB) a essa infraestrutura.

- **Passo a Passo Técnico:**

1. No seu terminal local, navegue até a pasta onde o código do PortifolioHUB está (ou onde será iniciado).
2. Caso já tenha um código local e não tenha clonado o repositório vazio, inicialize o Git localmente:

3. Vincule a sua pasta local ao repositório remoto do GitHub que você criou no Passo 1:
4. Certifique-se de que a branch padrão local seja nomeada como main (alinhada com o padrão atual do GitHub):
5. Faça o primeiro *pull* para alinhar os arquivos iniciais (README.md, .gitignore) gerados no GitHub com o seu ambiente local:

4.3- Como implementar o controle de versão com Git no dia a dia

Agora que o ambiente local do PortfolioHUB está conectado ao GitHub, você precisa adotar um fluxo de trabalho profissional para registrar as alterações de código, simulando o dia a dia de uma equipe de engenharia.

- **Estratégia de Ramificação (Feature Branch Workflow):**
Nunca desenvolva diretamente na branch main. Para cada nova funcionalidade do PortfolioHUB (como a integração com o Gemini ou um sistema de login), crie uma ramificação separada.
- **Fluxo Prático de Comandos:**
 1. Crie e mude para uma nova branch para trabalhar em uma funcionalidade:
 2. Após escrever o código de segurança ou funcional, verifique o status dos arquivos:
 3. Adicione as modificações ao estágio de preparação:
 4. Escreva um commit utilizando o padrão **Conventional Commits** (técnica que discutimos anteriormente para manter o histórico limpo):
 5. Envie a branch para o GitHub:

4.4- Como definir as práticas de colaboração (Governança do Código)

Com o código sendo versionado em branches separadas (Passo 3), você precisa estabelecer regras de colaboração para que essas alterações entrem na branch principal (main) de forma segura, garantindo a qualidade do software.

- **Implementação de Pull Requests (PRs):** No GitHub, abra um *Pull Request* da sua branch *feature/integracao-gemini* para a *main*. O PR serve como a mesa de discussão do código. Mesmo em um projeto solo, use o PR para fazer uma auto-revisão (*Self-Code Review*).
- **Regras de Proteção de Branch (Branch Protection Rules):** Vá em *Settings > Branches* no seu repositório do GitHub e proteja a branch *main*:
 - ✓ Exija Pull Requests antes do Merge: Bloqueie o push direto na *main*.
 - ✓ Exija aprovações: Em equipes, isso força outro engenheiro a revisar o código.
 - ✓ Exija que testes passem (CI/CD): Configure um *GitHub Action* para rodar seus testes automatizados e linters. O botão de *Merge* só será liberado se o código estiver íntegro e seguro.

4.5 Como documentar todo o processo no repositório

Como engenheiro de software, seu código só é tão bom quanto a sua documentação. O estágio final é consolidar toda a infraestrutura técnica (Passos 1 e 2), o fluxo de trabalho (Passo 3) e as regras de colaboração (Passo 4) em documentos acessíveis no próprio repositório.

- **Arquivos Essenciais de Documentação:**
 1. **README.md (A vitrine do projeto):** Deve conter o escopo do PortfolioHUB, como rodar o projeto localmente (configuração de variáveis *.env*), o stack tecnológico e os links para a documentação interna.

2. CONTRIBUTING.md (Guia de Colaboração Técnica): Pegue o modelo profissional de colaboração que estruturamos na conversa anterior e coloque-o neste arquivo. Ele explicará detalhadamente para outros desenvolvedores como usar as branches, o padrão de commits exigido e como abrir um PR válido.

3. SECURITY.md (Política de Segurança): Adicione as diretrizes de segurança que você gerou com o suporte da IA, detalhando como reportar vulnerabilidades e as boas práticas adotadas no código para proteger os dados dos portfólios.

Ao seguir esta sequência, você constrói o PortfolioHUB não apenas como um amontoado de código, mas como um produto de software real, estruturado sob as melhores práticas de engenharia

5.D- Finalização da integração e testes

5.1- Finalizando as integrações com o Google Gemini

- ✓ **Revise as integrações existentes:** Acesse o Google Gemini e descreva as integrações que você já implementou (Google Drive, Google Calendar, Git/GitHub). Solicite ao Gemini que analise suas implementações e sugira melhorias em termos de segurança, eficiência e usabilidade.
- ✓ **Implemente as integrações restantes:** Se você planejou integrar outras ferramentas do Google Workspace, como Gmail, Google Meet ou Google Keep, utilize o Gemini como guia para encontrar a melhor forma de fazê-lo.
- ✓ **Busque exemplos e tutoriais:** Solicite ao Gemini exemplos de código, tutoriais e documentação para as integrações que você precisa implementar.
- ✓ **Valide a compatibilidade:** Utilize o Gemini para verificar se as integrações que você implementou são compatíveis

com diferentes navegadores, dispositivos e sistemas operacionais.

5.2-Testando o PortifólioHub

- ✓ Crie um plano de testes: Defina os cenários de teste que você irá executar, incluindo testes de funcionalidade, usabilidade, segurança e desempenho
- ✓ Execute testes unitários: Teste cada componente do PortfolioHUB individualmente para garantir que ele funcione corretamente.
- ✓ Execute testes de integração: Teste a interação entre os diferentes componentes do
- ✓ PortfolioHUB e as integrações com o Google Workspace.
- ✓ Execute testes de sistema: Teste o PortfolioHUB como um todo, simulando o uso real da plataforma por diferentes usuários.
- ✓ Realize testes de carga e desempenho: Simule o acesso de múltiplos usuários simultaneamente para verificar a capacidade do PortfolioHUB de lidar com o tráfego.
- ✓ Corrija os erros encontrados: Documente os erros encontrados durante os testes e corrija-os antes do lançamento.

5.3 Últimos ajustes

- ✓ **Revise a documentação:** Certifique-se de que a documentação do PortfolioHUB está completa e atualizada, incluindo o guia de usuário, as políticas de segurança e as informações sobre as integrações.
- ✓ **Crie um ambiente de produção:** Configure um ambiente de produção separado do ambiente de desenvolvimento, onde o PortfolioHUB será hospedado e acessado pelos usuários.
- ✓ **Implante o PortfolioHUB no ambiente de produção:** Transfira o código-fonte e os arquivos do PortfolioHUB para o ambiente de produção.

- ✓ **Configure o domínio:** apontar Se você possui um domínio próprio, configure-o para para o ambiente de produção do PortfolioHUB.
- ✓ **Anuncie o lançamento:** Informe seus usuários sobre o lançamento do PortfolioHUB e como eles podem acessá-lo

5.4 Utilizando o Google Gemini como referência

Utilizando o Google Gemini como referência

Durante todo o processo de finalização, integração e testes, utilize o Google Gemini como referência para:

- ✓ **Encontrar soluções para problemas:** Se você encontrar algum problema durante a finalização da integração ou os testes, descreva o problema para o Gemini e peça ajuda para encontrar uma solução
- ✓ **Obter feedback sobre o design e a usabilidade:** Solicite ao Gemini feedback sobre o design e a usabilidade do PortfolioHUB, e implemente as sugestões de melhoria.
- ✓ **Gerar conteúdo para o lançamento:** Peça ao Gemini para gerar textos para o anúncio do lançamento, emails de boas-vindas para os usuários e posts para as redes sociais.

6.Revisão Final e apresentação

- **Funcionalidades:**

- ✓ Revise todas as funcionalidades do PortfolioHUB, garantindo que estejam
- ✓ funcionando como esperado e atendendo aos requisitos do projeto.
- ✓ Teste a integração com as ferramentas do Google Workspace (Drive, Calendar, etc.) e com o GitHub, verificando se a sincronização de dados, o controle de versão e o compartilhamento de código estão funcionando corretamente.

- ✓ Navegue pelas diferentes seções do PortfolioHUB, simulando o uso da plataforma por diferentes tipos de usuários, e corrija quaisquer erros ou inconsistências encontradas.

- **Conteúdo:**

- ✓ Revise todo o conteúdo textual do PortfolioHUB, como a descrição do projeto, o guia do usuário e as políticas de segurança, garantindo que as informações estejam claras, concisas e livres de erros gramaticais e ortográficos.
- ✓ Verifique se as imagens, vídeos e outros elementos visuais estão sendo exibidos corretamente e se contribuem para a experiência do usuário.
- ✓ Atualize o conteúdo do PortfolioHUB com as informações mais recentes sobre o projeto, como novas funcionalidades, atualizações e notícias relevantes.